

Ambiente de Execução

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

26 de novembro de 2024

Etapas de Compilação

- Até o momento, estudamos as três fases de análise de um compilador: análise léxica, análise sintática e análise semântica.
- Essas fases compõem a etapa de compilação denominada *front-end* e o seu principal objetivo é garantir que as definições da linguagem-fonte sejam respeitadas pelo programa-fonte.
- O produto do front-end é a representação intermediária do programa fonte (árvore abstrata, ou código de três endereços).
- Em seguida, vêm as fases de *geração de código*, e finalmente a *otimização de código*, que são denominadas em conjunto *back-end*.

- Para implementar o conjunto de abstrações da linguagem fonte: variáveis, escopo, parâmetros, funções, etc, o compilador cria um ambiente de execução, no qual assume que seus objetos serão executados.
- Esse ambiente de execução trata de uma série de questões como tais como a disposição em memória das diversas partes que formam um programa e mecanismos de acesso a variáveis, entre outros.
- Por exemplo, o compilador pode gerar código considerando que a disposição dos dados na memória obedece à política de pilha, conforme é mostrado no conjunto de slides que trata da geração de código.

- Abordamos os seguintes temas relacionados ao ambiente de execução:
 - Organização de memória
 - Registros de ativação e alocação de espaço na pilha.

Organização de memória para a execução de um programa

- Quando o código objeto de um programa é chamado (via shell), o SO reserva um espaço de endereçamento lógico para o programa executável.
- Cria um processo para executar o programa.
- Em seguida, o SO carrega o programa executável em parte da memória alocada e passa o controle de execução para o ponto de entrada do programa ("main()").
- O programa passa então a executar no modo usuário.
- A figura a seguir mostra a organização da memória típica de um programa em execução:

Subdivisão da Memória Usada por um Programa

Código	endereço alto
Dados Estáticos	
Heap	
Memória Livre	endereço baixo
Pilha	

Subdivisão da memória usada por um programa

Área de código e Área de Dados Estáticos

- A subdivisão da figura anterior é típica em um programa compilado, cuja linguagem-fonte é uma linguagem de terceira geração (Pascal, C) e linguagens orientadas a objeto.
- Algumas áreas na figura são estaticamente alocadas (em tempo de compilação):
 - Código do programa - o tamanho do código executável é fixo e pode ser determinado durante a carga do programa.
 - Memória Estática - geralmente as linguagens de programação possuem algum modo de definir variáveis estáticas e globais:
 - variáveis *static*
 - variáveis declaradas antes de qualquer função.
 - dados armazenados pelo compilador para coleta de lixo também são armazenadas nesta área.
 - variáveis locais e parâmetros de funções em linguagens que não possuem chamadas recursivas de procedimentos (Fortran)

Subdivisão da memória usada por um programa

Áreas Dinamicamente Alocadas

- Linguagens que possuem o conceito de bloco e chamadas recursivas de funções, necessitam representar o mesmo nome em posições distintas de memória. As alocações para essas memórias são definidas em tempo de execução.
- Estas variáveis são alocadas possivelmente várias vezes na área de memória denominada *pilha*.
- variáveis criadas com as operações do tipo *malloc* (C/C++) e *new* (Java/C++) também precisam ser alocadas dinamicamente. Essas variáveis são armazenadas no *heap*.

Algumas Considerações

- Nesse slide e nos seguintes fazemos algumas restrições sobre o ambiente de execução que vamos considerar para o trabalho. Entretanto, essas restrições não se aplicam quando há alguns recursos na linguagem-fonte (tratamento de exceções, programação concorrente, etc).
- A execução é sequencial, o controle muda de uma posição para outra no programa, em uma ordem bem definida.
 - Consideração não satisfeita quando a programação é concorrente: se uma ação vem antes da outra no código, não há garantia de qual é executada primeiro se a programação é concorrente.
- O controle da execução sempre retorna ao ponto imediatamente após a chamada de uma função.
 - Condição não satisfeita quando há tratamento de exceções na linguagem.

Ativação de Funções

- A chamada de uma função corresponde à *ativação* da função que passa a ser executada.
- O *tempo de vida* de uma ativação de uma função f é dado pela sequência de comandos de f , e das funções ativadas por f .
- Os tempos de vida de ativações de funções podem se sobrepor, isto é, se f chama q , a ativação de q coexiste com a ativação de f .
- Se uma função ativa duas funções x e y , nessa ordem, o tempo de vida de x termina antes de iniciar o de y .
- A sequência de ativações de funções de um programa pode ser representada por uma árvore, denominada *árvore de ativação*

Árvore de Ativação

Exemplo

```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```

main

Árvore de Ativação

Exemplo

```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```



Árvore de Ativação

Exemplo

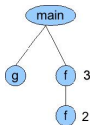
```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```



Árvore de Ativação

Exemplo

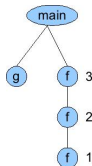
```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```



Árvore de Ativação

Exemplo

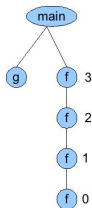
```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```



Árvore de Ativação

Exemplo

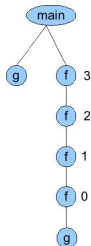
```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```



Árvore de Ativação

Exemplo

```
int g(){ return 1;}  
int f(int x) { if(x==0) g(); else f(x-1); }  
int main(){g(); f(3);}
```

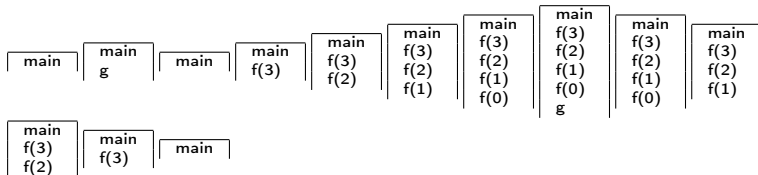
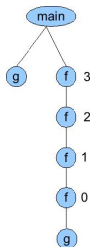


Simulação da Árvore de Ativação

- A ordem com que as funções são ativadas e desativadas, imposta pela árvore de ativação, pode ser simulada por uma pilha.
- Sempre que uma função é ativada, uma representação dessa ativação, denominada *registro de ativação* é empilhada.
- Um registro de ativação fica na pilha durante todo o tempo de vida da ativação de uma função.
- A sequência de empilhamentos e desempilhamentos de registro de ativação corresponde a uma simulação de um percurso em pré-ordem na árvore de ativação.

Controle de Ativações de Funções

Exemplo:



Registro de Ativação (Quadro ou *Frame*)

- Cada ativação viva de uma função tem um registro de ativação armazenado na pilha, contendo informações que permitem o gerenciamento de uma ativação.
- O RA de uma função f contém uma mistura de informações sobre a ativação de f e sobre o código p que ativou f . Essa mistura é necessária para completar a execução de f e para retomar à execução de p .
- A execução de p é suspensa após a chamada de f . Os registradores da máquina e a pilha passam a ser usados por f .
- Quando f termina, o contexto em que p estava em execução deve ser restaurado. Isto é, os valores dos registradores e da pilha devem voltar a ser os mesmos antes da chamada a f .

Controle de Ativações de Funções - RA

Estrutura de um RA

Resultado
Argumentos
Ligação de Controle
Endereço de Retorno

- A estrutura de RA acima não é padrão
- A estrutura depende da linguagem fonte, também da arquitetura para a qual o programa objeto está sendo gerado.
- Geralmente o valor retornado está na base do RA. Isso permite que o código da função chamadora (cujo RA está logo acima do RA da função chamado - pilha crescendo para baixo) possa ter acesso ao valor retornado quando retomar o controle da execução.

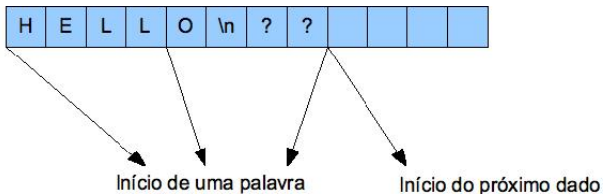
Uso do RA

- Compiladores comerciais tentam usar ao máximo registradores da máquina para armazenar dados do RA, em vez de armazená-los na pilha.
- A maioria das arquiteturas possuem registradores para armazenar um determinado número de argumentos de funções.
- Também há um registrador para endereço de retorno.
- O compilador determina em tempo de compilação a estrutura do RA e gera código que acessa e controla corretamente os RA.
- O RA para um compilador e a geração de código devem ser projetados em conjunto.

Uso do RA

- Os dados na memória de uma determinada arquitetura são *alinhados por palavra* se iniciam na memória a partir do início de uma palavra.
- É fundamental para um projetista de compilador conhecer o alinhamento de memória da máquina alvo.
- Em algumas arquiteturas, o uso de dados que não obedecem ao alinhamento adotado na arquitetura pode provocar mal funcionamento do programa.
- Em quase totalidade das arquiteturas, o uso incorreto do alinhamento causa lentidão de processamento.

Exemplo - alinhamento por palavra



- Os bytes marcados com ? na figura não são utilizados.
- O próximo dado a ser armazenado inicia na próxima palavra disponível na memória.